

push_swap Project Guide

Technical code documentation - structure, functions, and modification guide

This document explains every file and every function in the project, how the files fit together, and, most importantly, where to look in the code when you want to understand or change a specific behavior.

42 project - **push_swap**. A program that sorts a stack of integers using two stacks (**a** and **b**) and a limited set of operations (sa, sb, ss, pa, pb, ra, rb, rr, rra, rrb, rrr).

Contents

- 1. Overview and execution flow
- 2. Data structures (push_swap.h)
- 3. Entry point: main.c
- 4. Argument parsing (parse*.c)
- 5. The stack: stack.c
- 6. Core operations: ops_*.c
- 7. Strategy selection: sort_mode.c
- 8. Simple strategy: sort_small.c
- 9. Medium strategy (chunks): sort_chunk.c
- 10. Complex strategy (radix): sort_radix.c
- 11. Sorting utilities: sort_utils.c
- 12. Disorder calculation: disorder.c
- 13. Benchmark mode: bench*.c
- 14. Error handling: error.c
- 15. Generic utilities: utils.c
- 16. Practical guide: "I want to change X"

1. Overview and execution flow

The program receives a list of numbers as arguments, loads them into stack **a**, and then sorts them using auxiliary stack **b** and the permitted operations. Every operation performed is printed to stdout (for example: sa, pb...).

Execution order (main.c)

```
main()
|-- parse_config()      -> reads flags (--simple/--medium/--complex/--adaptive/--bench)
|-- parse_arguments()  -> converts argv into an integer array (t_parse_result)
`-- run_program()
    |-- bench_init()    (when --bench is enabled)
    |-- stack_init()    -> initializes stack a (values) and b (empty)
    |-- compute_disorder() -> calculates the disorder of stack a
    |-- run_sort()      -> selects and launches the sorting strategy
    |   |-- sort_small() (simple mode)
    |   |-- sort_chunk() (medium mode)
    |   |-- sort_radix() (complex mode)
    |-- bench_print()   (when --bench is enabled)
    `-- stack_free()    -> releases memory
```

Key point: The default mode is **MODE_ADAPTIVE**: the program calculates the stack's disorder and selects simple, medium, or complex automatically according to the thresholds defined in **sort_mode.c**.

File map by responsibility

File	Function	Responsibility
main.c	main()	Entry point; orchestrates the entire program
parse.c / parse_mode.c / parse_split.c / parse_int.c	parse_*	Reads and validates arguments
stack.c	stack_*	Initialization, cleanup, and sorted-state check
ops_*.c	op_*	The 11 allowed operations
sort_mode.c	run_sort()	Chooses which strategy to use
sort_small.c	sort_small()	Simple $O(n^2)$ strategy
sort_chunk.c	sort_chunk()	Medium $O(n \cdot \sqrt{n})$ strategy
sort_radix.c	sort_radix()	Complex $O(n \log n)$ strategy
sort_utils.c	compress_values(), index_of_max()	Functions shared by sorting strategies
disorder.c	compute_disorder()	Measures how disordered the stack is
bench.c / bench_report.c / bench_counts.c	bench_*	--bench execution statistics
error.c	error_exit(), parse_error_exit()	Error handling and clean exit
utils.c	ft_strlen, ft_strcmp, ft_memmove...	Small generic utility functions

2. Data structures - push_swap.h

This central header contains every structure and prototype in the project. It is the first file to consult when learning the code's vocabulary.

t_mode - the four strategies

```
typedef enum e_mode
{
    MODE_SIMPLE,
    MODE_MEDIUM,
    MODE_COMPLEX,
    MODE_ADAPTIVE
} t_mode;
```

Represents the selected sorting mode, either specified by a flag or chosen automatically in adaptive mode.

t_config - command-line configuration

```
typedef struct s_config
{
    t_mode mode; // selected strategy (--simple, --medium, ...)
    int bench; // 1 when --bench is present, otherwise 0
} t_config;
```

t_bench - operation counters

A variable of this type contains one counter for each of the 11 operations (sa, sb, ss, pa, pb, ra, rb, rr, rra, rrb, rrr). It is used only in **--bench** mode.

t_stack - a stack

```
typedef struct s_stack
{
    int *arr; // dynamic value array (arr[0] is the top)
    int size; // current number of elements in the stack
    t_bench *bench; // benchmark counters (NULL without --bench)
} t_stack;
```

Key point: The stack is not a linked list. It is a plain array in which **arr[0]** represents the top. Every operation (rotate, push, etc.) therefore moves array elements physically.

t_parse_result and t_program

t_parse_result contains the integer array extracted from the arguments and its element count. **t_program** groups the full program state (configuration, parsing result, stack a, stack b, and benchmark data); it is the structure passed around in main.c.

3. Entry point - main.c

File	Function	Responsibility
main.c	get_bench_ptr()	Enables benchmarking when --bench is present; otherwise returns NULL
main.c	init_stacks()	Initializes stack a with values and stack b empty, with the same capacity
main.c	run_program()	Orchestrates benchmark -> initialization -> disorder -> sort -> report -> cleanup
main.c	main()	Checks argc, calls parse_config and parse_arguments, then launches run_program

Details

main(): when fewer than two arguments are provided, the program exits silently because there is nothing to sort. Otherwise it first parses flags (--simple, etc.) with **parse_config**, then parses the numbers with **parse_arguments**. If either step fails, **parse_error_exit()** prints "Error" and exits.

run_program(): calculates disorder before sorting, using the original (uncompressed) values, then calls **run_sort()**, which performs the actual work.

Key point: To change the overall step order - for example, to add another validation step or change when benchmarking is initialized - edit **run_program()** here.

4. Argument parsing

Four files work together to transform argv into a validated list of integers.

parse_mode.c - reading flags (--simple, --bench, etc.)

File	Function	Responsibility
parse_mode.c	apply_flag()	Maps a string ("--simple", etc.) to a t_config field
parse_mode.c	parse_config()	Loops over argv while entries begin with "--" and consumes recognized flags

parse_config() advances *argv and decrements *argc for each recognized flag. After it runs, argv/argc contain only the program name and the numbers to sort.

Key point: To add a new flag such as --verbose, edit **apply_flag()** in parse_mode.c and add the corresponding field to **t_config** in push_swap.h.

parse.c - building the integer list

File	Function	Responsibility
parse.c	has_duplicate()	Checks whether a value already exists; duplicates are forbidden
parse.c	grow_values()	Doubles the capacity of the dynamic value array
parse.c	append_value()	Adds a non-duplicate value and grows the array if necessary
parse.c	parse_one_arg()	Splits one argument into tokens and parses each token as an integer
parse.c	parse_arguments()	Loops over all remaining argv entries and builds t_parse_result

A single argument may contain several whitespace-separated numbers (for example: ./push_swap "3 2 1"). Therefore, parse_one_arg first splits it with **split_whitespaces** (see parse_split.c).

parse_split.c - whitespace splitting (a small split())

File	Function	Responsibility
parse_split.c	count_words()	Counts words separated by whitespace
parse_split.c	dup_word()	Copies a substring into newly allocated memory
parse_split.c	fill_words()	Fills the word array from the source string
parse_split.c	split_whitespaces()	Public function: a home-made split on whitespace

parse_int.c - strict string-to-int conversion

File	Function	Responsibility
parse_int.c	is_digit()	Checks whether a character is a digit from 0 to 9
parse_int.c	out_of_range()	Detects overflow outside the signed 32-bit int range
parse_int.c	parse_sign()	Reads an optional leading + or - sign
parse_int.c	parse_int_strict()	Converts a string to int and rejects anything that is not a valid number

Key point: "Strict" means no extra characters after the number and no INT_MIN/INT_MAX overflow. Check this function when an unusual input is not rejected as expected, such as "12abc", "999999999999", or " 42 ".

5. The stack - stack.c

File	Function	Responsibility
stack.c	stack_init()	Assigns arr, size, and bench to the structure; no allocation happens here
stack.c	stack_free()	Frees arr and resets the structure to zero
stack.c	stack_is_sorted()	Returns 1 when the array is sorted in ascending order, otherwise 0

These three functions are deliberately simple: stack.c contains no sorting logic. It only manages the basic life cycle of the **t_stack** structure.

6. Core operations - ops_*.c

These files implement the 11 official push_swap operations. Every public operation has the same final parameter: an integer named **print**. When true, it prints the operation name to stdout and increments the corresponding benchmark counter.

ops_swap.c - sa, sb, ss

File	Function	Responsibility
ops_swap.c	swap_top()	Swaps the first two elements of a stack; static and reused
ops_swap.c	get_bench()	Returns the benchmark pointer from a or b, whichever is non-NULL first
ops_swap.c	op_sa()	Swaps the first two elements of stack a
ops_swap.c	op_sb()	Swaps the first two elements of stack b
ops_swap.c	op_ss()	Performs sa and sb at the same time

ops_push.c - pa, pb

File	Function	Responsibility
ops_push.c	push_top()	Removes the top of src and inserts it at the top of dst, shifting all elements
ops_push.c	op_pa()	Pushes the top of b onto a
ops_push.c	op_pb()	Pushes the top of a onto b

Key point: push_top() shifts every array element on each call ($O(n)$). This is the most expensive part in terms of memory movement. To optimize CPU performance - rather than the number of printed operations - consider a different data structure here, such as a linked list or circular index.

ops_rotate.c - ra, rb, rr

File	Function	Responsibility
ops_rotate.c	rotate()	Moves the first element to the last position (upward rotation)
ops_rotate.c	get_bench()	Returns the benchmark pointer from a or b
ops_rotate.c	op_ra()	Calls rotate() on a
ops_rotate.c	op_rb()	Calls rotate() on b
ops_rotate.c	op_rr()	Rotates a and b at the same time

ops_reverse_rotate.c - rra, rrb, rrr

File	Function	Responsibility
ops_reverse_rotate.c	reverse_rotate()	Moves the last element to the first position (reverse rotation)
ops_reverse_rotate.c	get_bench()	Returns the benchmark pointer from a or b
ops_reverse_rotate.c	op_rra()	Calls reverse_rotate() on a
ops_reverse_rotate.c	op_rrb()	Calls reverse_rotate() on b
ops_reverse_rotate.c	op_rrr()	Reverse-rotates a and b at the same time

Key point: All operations are independent of the sorting strategies. If output or benchmark counting must change for every operation, work in these three **ops_*.c** files - never in the **sort_*.c** files.

7. Strategy selection - sort_mode.c

This is the orchestrator that decides which sorting strategy to use.

File	Function	Responsibility
sort_mode.c	select_adaptive()	Chooses simple, medium, or complex according to the disorder ratio
sort_mode.c	launch_sort()	Calls sort_small(), sort_chunk(), or sort_radix() according to the mode
sort_mode.c	run_sort()	Public function: resolves MODE_ADAPTIVE when needed, then launches sorting

Adaptive-mode thresholds

```
disorder < 0.2      -> MODE_SIMPLE
0.2 <= disorder < 0.5 -> MODE_MEDIUM
disorder >= 0.5     -> MODE_COMPLEX
```

Key point: This is **the** file to edit when changing adaptive-mode thresholds or adding a fifth strategy: update **select_adaptive()** for thresholds and **launch_sort()** to connect the new sorting function.

8. Simple strategy - sort_small.c ($O(n^2)$)

This strategy is used for small stacks or a low disorder ratio (below 20%).

File	Function	Responsibility
sort_small.c	index_of_min()	Finds the index of the smallest value in a
sort_small.c	sort_three()	Sorts exactly three elements with a fixed sequence of sa/ra/rra
sort_small.c	sort_small()	Public function: repeatedly extracts the minimum to b, then rebuilds a

How sort_small() works

While a->size > 3:

1. find the index of the minimum
2. bring it to the top (ra if near the start, rra if near the end)
3. pb (push this minimum onto b)

When three elements or fewer remain in a:

-> sort_three() puts them in order

Then push everything back from b to a with pa

(b contains the previously extracted minimum values; pushing them back in this order reconstructs the sorted stack)

Key point: To change behavior for very small arrays of two or three elements, inspect **sort_three()**. It is a hard-coded sequence of comparisons for the six possible permutations of three values.

9. Medium strategy - sort_chunk.c ($O(n \cdot \sqrt{n})$)

This chunk-based strategy is used for a medium disorder ratio (20% to 50%).

File	Function	Responsibility
sort_chunk.c	int_sqrt()	Integer square root without
sort_chunk.c	bring_max_to_top()	Brings the maximum in b to the top using rb or rrb according to distance
sort_chunk.c	push_chunk()	Pushes to b all values in the inclusive range [lower, upper]
sort_chunk.c	fill_chunks()	Splits [0, n-1] into chunks and calls push_chunk for each one
sort_chunk.c	sort_chunk()	Public function: compresses values, builds chunks, then pushes maxima back

How sort_chunk() works

1. compress_values(a) -> replace values with ranks (0..n-1)
2. chunk_size = sqrt(n)
3. For each [lower, upper] range of chunk_size elements:
 push_chunk() moves the values in that range from a to b
4. While b is not empty:
 find b's maximum, bring it to the top, then pa
 (largest chunks return first, rebuilding ascending order in a)

Key point: Chunk size ($\text{chunk_size} = \sqrt{n}$) is the key parameter of this strategy. To tune it, for example to reduce the number of operations, edit `int_sqrt()` or the formula in `sort_chunk()`.

10. Complex strategy - sort_radix.c ($O(n \log n)$)

This bit-by-bit radix sort works on compressed ranks and is used for high disorder (50% or more).

File	Function	Responsibility
sort_radix.c	get_max_bits()	Calculates the number of bits needed to represent n-1
sort_radix.c	process_bit()	One pass: sends bit-0 values to b and keeps bit-1 values in a via ra
sort_radix.c	sort_radix()	Public function: compresses values and calls process_bit once per bit

How sort_radix() works

1. compress_values(a) -> ranks 0..n-1
 2. For bit = 0 through max_bits - 1:
 - For every element (n times):
 - if bit `bit` of arr[0] is 0 -> pb (move it to b)
 - otherwise -> ra (move it to the bottom of a)
 - After all n elements are processed, push everything from b to a (pa)
- > after every bit has been processed, a is sorted

Key point: This is a classic LSB radix sort adapted to stacks. If a specific case does not sort correctly, first check **get_max_bits()** (the number of passes), then trace **process_bit()** one bit at a time.

11. Sorting utilities - sort_utils.c

Functions shared by several strategies, especially medium and complex.

File	Function	Responsibility
sort_utils.c	sort_ints()	Simple bubble sort on a raw array; used internally, not on the stack
sort_utils.c	find_index()	Finds a value's index in a sorted array
sort_utils.c	compress_values()	Replaces every stack value with its rank (0..n-1)
sort_utils.c	index_of_max()	Finds the index of the largest value in a stack

Key point: compress_values() is essential to the medium and complex strategies. It allows them to work with ranks (0, 1, 2...) instead of actual values, simplifying chunk and radix-bit calculations.

12. Disorder calculation - disorder.c

File	Function	Responsibility
disorder.c	compute_disorder()	Calculates the ratio of inverted pairs to total pairs

For every pair (i, j) where $i < j$, the function counts an inversion when $a[i] > a[j]$. The result is a ratio from 0.0 (already sorted) to 1.0 (completely reversed).

Key point: This value feeds `select_adaptive()` in `sort_mode.c`. If you change the formula here, the adaptive thresholds of 0.2 and 0.5 may also need recalibration.

13. Benchmark mode - bench.c / bench_report.c / bench_counts.c

File	Function	Responsibility
bench.c	bench_init()	Sets every t_bench counter to 0
bench_counts.c	write_str(), write_num()	Low-level output to stderr without printf
bench_counts.c	print_count()	Prints "name: value" for one operation
bench_counts.c	bench_print_counts()	Prints all 11 operation counters
bench_report.c	write_str(), write_num()	The same output helpers, duplicated in this file
bench_report.c	bench_total()	Adds all counters to obtain the total operation count
bench_report.c	print_strategy()	Prints the strategy used and its theoretical complexity
bench_report.c	bench_print()	Public function: prints disorder %, strategy, total, and every counter

Key point: Counters are incremented directly inside the **op_*()** functions in ops_*.c, through t_stack's bench field, and only when **print** is true. To add a statistic: 1) add a field to t_bench, 2) initialize it in bench_init(), 3) increment it in the correct ops_*.c location, and 4) display it in bench_print_counts().

14. Error handling - error.c

File	Function	Responsibility
error.c	parse_error_exit()	Writes "Error\n" to stderr and exits with code 1 after a parsing error
error.c	error_exit()	Frees non-NULL stacks a/b, writes "Error\n", and exits with code 1

These two functions are the program's only error exit points. **error_exit()** is used after stack initialization, so it releases memory that has already been allocated. **parse_error_exit()** is used earlier, when no stack memory has been allocated yet.

15. Generic utilities - utils.c

File	Function	Responsibility
utils.c	is_space()	True when the character is a space, tab, or line break
utils.c	ft_strlen()	Equivalent to strlen
utils.c	ft_strcmp()	Equivalent to strcmp
utils.c	ft_memmove()	Equivalent to memmove; safe even when memory areas overlap
utils.c	free_tokens()	Frees a NULL-terminated array of strings (char **)

This file contains no business logic. Its functions are classic libc reimplementations used by other files, especially during parsing.

16. Practical guide - "I want to change..."

I want to...	Where to look
Change adaptive-mode thresholds (0.2 / 0.5)	sort_mode.c, function select_adaptive().
Change the disorder formula	disorder.c, function compute_disorder().
Add a command-line flag (--xxx)	parse_mode.c (apply_flag()) and a new field in t_config (push_swap.h).
Change how duplicates or invalid numbers are rejected	parse.c (has_duplicate(), append_value()) and parse_int.c (parse_int_strict()).
Change how three elements are sorted	sort_small.c, function sort_three().
Change medium-strategy chunk size	sort_chunk.c, function int_sqrt(), or the chunk_size = int_sqrt(n) line in sort_chunk().
Understand or change radix pass count	sort_radix.c, function get_max_bits().
Add an allowed operation	Add its prototype to push_swap.h, create ops_xxx.c based on ops_swap.c, and add it to COMMON_SRCS in the Makefile.
Change error messages or behavior	error.c (error_exit() / parse_error_exit()).
Add a --bench statistic	1) t_bench, 2) bench_init(), 3) increment it in the right ops_*.c file, 4) display it in bench_print_counts().
Change benchmark report formatting	bench_report.c, function bench_print().
Find where operations are counted and printed	Inside each op_*() function in ops_*.c; each one manages its own write() and benchmark counter.
Change the stack structure (for example, use a linked list)	t_stack in push_swap.h, all op_*() functions in ops_*.c (they access arr[] directly), and stack.c.

Compilation chain reminder

The Makefile divides sources into two groups: **COMMON_SRCS** (parsing, operations, utilities, benchmarking, etc.) and **PUSH_SRCS** (main.c plus the three sorting strategies). When you add a new .c file, remember to put it in the correct Makefile list; otherwise it will not be compiled.